The Ackermann Function and the Turing Machine: The Canonical Boundary of Primitive Recursion

For readers working in formal languages, mathematical logic, and the foundations of computability.

Abstract

The Ackermann function is the canonical witness to the strict inclusion

$$\{\text{primitive recursive functions}\} \subsetneq \{\text{total Turing-computable functions}\}.$$

It is total and effectively computable — a Turing machine evaluates it on every input and halts — yet it is not primitive recursive. We make this precise in three equivalent guises: (i) in the *function* sense, the diagonal $n \mapsto A(n,n)$ is not primitive recursive, because it escapes every fixed row of the scale that exhausts the primitive-recursive functions; (ii) in the *resource* sense, **no Turing machine computing A has a primitive-recursive bound on its running time**; and (iii) in the *proof-theoretic* sense, A is provably total in Peano Arithmetic but sits above the fragment that proves the totality of all primitive-recursive functions. The mechanism is a *scale that exhausts* the primitive-recursive functions — equivalently, the **fast-growing hierarchy** $(F_m)_{m\in\mathbb{N}}$ or, more cleanly, the family of Ackermann *rows* $A(k,\cdot)$: every primitive-recursive function is dominated by one *fixed* element of the scale, whereas the diagonal of A has its scale-index equal to its input and therefore climbs the scale without bound. We give full proofs of the domination lemma and the non-primitive-recursiveness of A, and situate the result in the theory of provable totality (ordinal ε_0) and the Kirby–Paris hydra interpretation.

---

1. Thesis and roadmap

A reader might ask: *in what sense* does the Ackermann function "relate to" a Turing machine? The naive answer — "a Turing machine can compute it" — is true but vacuous, since every total computable function is Turing-computable by definition. The substantive relationship is that A is the **canonical boundary object** between two classes that a Turing machine straddles:

$$\underbrace{\text{PR}}_{\text{provably PR-bounded computation}} \subsetneq \underbrace{\text{total computable}}_{\text{halts, but no PR time bound}} = \text{Turing-computable} = \mu\text{-recursive}.$$

We use "canonical" deliberately, not "minimal" or "the immediate successor": in the partial order of total functions under pointwise domination there is a dense zoo of total recursive functions strictly between the PR functions and A (and even below A on some inputs), so A is not the least total-computable function that escapes the PR class. What makes A the canonical boundary is that it is the *first natural, simply-defined* function to do so — the standard textbook witness to the strictness of the inclusion, and (as we show) one for which no PR time bound can certify the computation.

Concretely we establish:

1. A **is Turing-computable.** Its recursive definition is effective and terminates; we exhibit the machine and the μ-recursive construction (§3).
2. A **is not primitive recursive** (§6). The proof uses a *scale that exhausts* the primitive-recursive functions (the Ackermann rows, equivalently the fast-growing hierarchy):

every PR function is dominated by one fixed element of the scale, whereas the diagonal $n \mapsto A(n,n)$ has its scale-index equal to its input and outruns every fixed element.

3. **The resource reading.** f is PR iff some Turing machine computes f within a PR time bound (§4). Hence A being non-PR is *equivalent* to: every Turing machine computing A runs for more than any PR function of its input. A is the first total function a Turing machine can compute that no PR bound can certify.
4. **The proof-theoretic reading.** A is provably total in PA; the provably-total functions of PA are exactly those dominated by the fast-growing hierarchy up to ordinal ε_0 (§7). A's growth is a finite level of that hierarchy.

We fix notation in §2, develop the exhaustion scale in §5, and close with the hydra interpretation and open directions.

---

2. The Ackermann–Péter function

We use the two-argument (Péter) form, the standard one in the literature on primitive recursion:

$$\begin{aligned} A(0,n) &= n+1, \\ A(m+1,0) &= A(m,1), \\ A(m+1,n+1) &= A(m,\, A(m+1,n)). \end{aligned} \tag{1}$$

(Historical note. Ackermann (1928) introduced a related three-argument function in his study of a gap in Hilbert's finitist programme; Péter (1956–58) recast it as (1) and proved it is not primitive recursive. The name "Ackermann function" now refers to (1).)

2.1 The first rows: one iteration level per row

Unfolding (1) gives closed forms for the low rows. Each row is exactly one *iteration level* above the previous:

m	$A(m,n)$	operation
0	$n+1$	successor
1	$n+2$	addition (from 2)
2	$2n+3$	multiplication
3	$2^{\,n+3}-3$	exponentiation
4	$2{\uparrow\uparrow}\,(n+3)-3$	tetration (power tower)

(2)

where $\uparrow\uparrow$ is Knuth's double arrow: $a\uparrow\uparrow 1 = a$ and $a\uparrow\uparrow(k+1) = a^{\,a\uparrow\uparrow k}$.

Proof sketch (by induction on m). $m=0$ is immediate. Suppose $A(m,n) = 2n+3$. Then $A(m+1,0) = A(m,1) = 2\cdot 1+3 = 5 = 2^{1+3}-3$, and $A(m+1,n+1) = A(m, A(m+1,n)) = 2\,A(m+1,n)+3 = 2(2^{n+3}-3)+3 = 2^{n+4}-3$, so $A(m+1,n) = 2^{n+3}-3$. The $m=3 \to 4$ step is identical in shape but replaces the linear recurrence $x \mapsto 2x+3$ by $x \mapsto 2^x-3$, yielding a power tower: $A(4,n+1) = 2^{\,A(4,n)}-3$ with $A(4,0) = 13 = 2\uparrow\uparrow 3-3$, hence $A(4,n) = 2\uparrow\uparrow(n+3)-3$. □

The pattern is the intuitive engine of everything that follows: **each increment of the first argument adds one level of iteration** (successor $\to$ $+$ $\to$ $\times$ $\to$ $\to\uparrow\uparrow$), which is precisely what the fast-growing hierarchy formalizes.

The growth is not merely fast — it is *unwritable* within a few rows. Verified by direct computation:

- $A(4,1) = 65\,533$;
- $A(4,2) = 2 \uparrow\uparrow 5 - 3 = 2^{65536} - 3$, an integer with **19 729** decimal digits;
- the diagonal value $a(4) = A(4,4) = 2 \uparrow\uparrow 7 - 3$ is a power tower of seven 2's; its *decimal digit count* is itself the 19 728-digit integer $\approx 6.03 \times 10^{19\,727}$.

2.2 Totality

A is **total**: $A(m,n)$ is defined for all $m, n \in \mathbb{N}$.

Proof (by induction on m). $A(0,n) = n+1$ is total. Assume $A(m,\cdot)$ is total. We show $A(m+1,\cdot)$ is total by induction on n: $A(m+1,0) = A(m,1)$ is defined by the inductive hypothesis on m; and $A(m+1,n+1) = A(m, A(m+1,n))$ is defined because $A(m+1,n)$ is (inner induction) and $A(m,\cdot)$ is total (outer induction). □

This is a genuine, constructive termination argument — not an appeal to a completeness theorem — and it is what licenses the Turing machine of §3.

---

3. A is Turing-computable

3.1 The machine

We describe a deterministic one-tape Turing machine M_A that, on input the pair (m,n) (unary or binary), halts with $A(m,n)$. The tape plays two roles: a **stack of pending frames** (m', n') and a **work area** for the current value.

M_A pushes the frame (m,n) and loops:

1. Pop the top frame (m', n').
2. If $m' = 0$: the value is $n' + 1$; deliver it (to the continuation frame below, or as the final output).
3. If $n' = 0$: push $(m' - 1, 1)$ [the clause $A(m+1,0) = A(m,1)$].
4. If $n' > 0$: push a *continuation* frame $(m' - 1, \cdot)$ and then push $(m',\ n' - 1)$. The machine first computes $A(m', n' - 1)$; when that value v is returned, it pushes $(m' - 1, v)$, thereby computing $A(m' - 1, v) = A(m', n')$ [the clause $A(m+1, n+1) = A(m, A(m+1,n))$].

Because A is total (§2.2), the stack is always eventually exhausted, so M_A halts on every input. Finite control plus a tape-as-stack is a legitimate Turing machine, so A **is Turing-computable**.

3.2 The μ-recursive reading

Equivalently, A is a **general (μ-)recursive function**. The recursion (1) is *not* a primitive recursion: in the clause $A(m+1, n+1) = A(m, A(m+1,n))$ the argument of the outer call is itself the value of a recursive sub-computation, so the *depth* of the recursion on an input (m,n) is a function of the *computed values*, not a fixed primitive-recursive function of the input. This is a purely syntactic observation about the *shape* of the definition — the recursion is *general* (its unfolding depth is data-dependent) rather than *primitive* (whose unfolding depth is bounded a priori). It is, however, a *well-founded* definition: §2.2 proves by double induction that every sub-computation is eventually evaluated, so the recursion terminates on every input. By the fundamental equivalences of the 1930s —

$$\text{general recursive} \;=\; \mu\text{-recursive} \;=\; \text{Turing-computable} \;=\; \lambda\text{-definable}$$

(Kleene 1936; Church 1936; Turing 1936) — A is computed by a Turing machine. The point of §6 is that A lies in this class **but not** in the primitive-recursive sub-fragment; in particular, the data-dependent depth is *not* bounded by any primitive-recursive function of (m,n) (Corollary 2), which is exactly the resource-theoretic content of Theorem 2.

---

4. The primitive-recursive functions, and their exact Turing-machine characterization

A function is **primitive recursive (PR)** iff it belongs to the smallest class containing the zero function, the successor $S(x) = x + 1$, and the projections $\pi_i^k(x_1, \dots, x_k) = x_i$, and closed under **composition** and **primitive recursion**:

$$f(x, 0) = g(x), \qquad f(x, t+1) = h(x, t, f(x, t)) \quad \Longrightarrow \quad f \text{ is PR if } g, h \text{ are.}$$

The key to the whole essay is the following bridge between the syntactic class "PR" and the operational class "Turing machine with a PR clock".

> **Theorem 1 (PR = PR-bounded-time Turing-computable).** A function $f : \mathbb{N}^k \to \mathbb{N}$ is primitive recursive iff there exist a deterministic one-tape Turing machine M computing f and a primitive-recursive function $t : \mathbb{N} \to \mathbb{N}$ such that M halts within $t(|x|)$ steps on every input x.

Proof.

(⇒) By structural induction on f. The initial functions are computed by trivial machines with obvious (linear, hence PR) step bounds. Composition: run the machines for the sub-functions in sequence; the step bound is the sum (a PR function) of the sub-bounds. Primitive recursion: $f(x, t)$ is computed by a machine that simulates the step h exactly t times; since t is part of the input and h has a PR step bound, the total bound is PR (a PR-bounded loop). Every PR function is thus computed by a machine whose running time is PR-bounded.

(⇐) Conversely, suppose M computes f and halts within $t(|x|)$ steps with t PR. First, the standard coding: the configuration of a one-tape machine at any instant is the finite datum (state, head position, finite non-blank tape contents, and the two head-adjacent symbols), which is encoded as a single natural number by any Gödel coding; under this coding the one-step transition $\mathrm{cfg} \mapsto T(\mathrm{cfg})$ is a **primitive-recursive** (indeed Δ_0) function, because it is a finite case analysis over the finite transition table. Make the halting states *absorbing* (T fixes a halting configuration), so that the s-fold iterate is defined for all s. Define the iterate **explicitly by primitive recursion**:

$$g(y, 0) = y, \qquad g(y, s+1) = T(g(y, s)).$$

Since T is PR, g is PR in (y, s). The configuration after s steps is $\mathrm{cfg}_s = g(\mathrm{cfg}_0, s)$, which is therefore PR in (cfg_0, s) — this is the content of "bounded iteration of a PR function is PR." Taking $s = t(|x|)$ (a PR value) and reading the output from $\mathrm{cfg}_{t(|x|)}$ (which equals the halting configuration, since M halted by then) yields f as a PR function. □

Corollary 1. f is *not* primitive recursive iff **no** Turing machine computing f has a primitive-recursive time bound.

This corollary is the resource-theoretic translation of everything that follows: once we prove A is not PR, it is automatic that every Turing machine computing A (including M_A of §3) runs for more than any PR function of its input.

---

5. Two scales that exhaust the primitive-recursive functions

5.1 The fast-growing hierarchy

Define the (finite-level) **fast-growing hierarchy** $(F_m)_{m \in \mathbb{N}}$ by

$$F_0(n) = n+1, \qquad F_{m+1}(n) = F_m^{\circ n}(n), \tag{3}$$

i.e. $F_{m+1}(n)$ is the n-fold iterate of F_m, evaluated at n. Unfolding (3) gives

$$F_0(n) = n+1, \quad F_1(n) = 2n, \quad F_2(n) = 2^n\, n, \quad F_3(n) \approx 2\uparrow\uparrow n, \quad F_4(n) \approx \text{ a tower of 2's of height } F_3(n$$

so, in terms of power towers of 2's, $F_2(n)$ is a tower of height 2, $F_3(n)$ is a tower of height $\sim n$ (tetration), and for $m \geq 4$ the tower height of $F_m(n)$ is $\sim F_{m-1}(n)$ — the tower height itself climbs the hierarchy. (Note $F_1(n) = 2n$ and $F_2(n) = 2^n n$: the latter is *exponential times linear*, not quadratic — a point that matters below, since the naive $2n^2$ mis-estimates the level of $A(3,\cdot)$.)

Lemma 1 (PR-closure). The function $\Phi(m,n) = F_m(n)$ is primitive recursive.

Proof. By induction on m. $\Phi(0,n) = n+1$ is PR. For the step, $\Phi(m+1,n) = F_m^{\circ n}(n)$: define $I(m,n,s)$ by $I(m,n,0) = n$ and $I(m,n,s+1) = F_m(I(m,n,s)) = \Phi(m, I(m,n,s))$. The body is PR by the inductive hypothesis, so I is PR by primitive recursion in s, and $\Phi(m+1,n) = I(m+1,n,n)$ is PR by composition. □

5.2 The domination lemma (Ackermann-row form)

The scale that does the work in the proof is the family of Ackermann *rows* $A(k,\cdot)$. Two elementary inequalities — both proved by induction on the second argument and verified by direct computation — are all that is needed.

(I) Level-additivity. For all a, b, n,

$$A(a,\, A(b,n)) \;\leq\; A(a+b+1,\, n). \tag{4}$$

(II) Iteration bound. Let T be a monotone function with $T(z) \leq A(b,\, z+C)$ for a constant C (independent of z). Then its t-fold iterate from any s satisfies

$$T^{\circ t}(s) \;\leq\; A(b+1,\, s+t+C'), \tag{5}$$

for a constant C' independent of s, t. (The special case $T(z) = A(b,z)$, $C = 0$ is the one checked numerically: $A_b^{\circ t}(s) \leq A(b+1, s+t)$.)

Intuition for (I)–(II): (I) says composing two rows costs at most the *sum* of their indices plus one — a row of Ackermann is a *level* of iteration, and levels add under composition. (II) says that iterating a row-b-bounded map t times climbs to row $b+1$, with the argument tracking the number of iterations — exactly the shape of a primitive recursion.

Lemma 2 (PR functions are dominated by a fixed Ackermann row). Every primitive-recursive $f : \mathbb{N}^k \to \mathbb{N}$ satisfies

$$f(x_1, \ldots, x_k) \;\leq\; A(K,\, \langle x \rangle) \qquad \text{for all } x, \tag{6}$$

for some fixed K (depending only on f), where $\langle x \rangle = \sum_i x_i$ is a fixed PR size.

Proof (structural induction on f).

- **Initial functions.** $S(x) = x+1 = A(0,x) \leq A(1, \langle x \rangle)$; a projection $x_i \leq \langle x \rangle \leq A(1, \langle x \rangle)$; a constant $c \leq A(1, \langle x \rangle)$ eventually. So $K = 1$ suffices.
- **Composition.** Let $f = h(f_1, \ldots, f_r)$, with $f_i(x) \leq A(k_i, \langle x \rangle)$ and $h(y) \leq A(K, \langle y \rangle)$ by induction; put $k_{\max} = \max_i k_i$. Then $f_i(x) \leq A(k_{\max}, \langle x \rangle)$, so by monotonicity of h

$$f(x) = h(f_1(x), \ldots, f_r(x)) \;\leq\; A(K,\, \textstyle\sum_i f_i(x)) \;\leq\; A(K,\, r \cdot A(k_{\max}, \langle x \rangle)).$$

A fixed multiplicative factor is absorbed by a bounded number of levels: for each fixed r there is a constant $c = c(r)$ with $r \cdot A(k_{\max}, n) \leq A(k_{\max} + c, n)$ eventually (a tower

of height $k_{\max}+c$ dominates any fixed multiple of the row $A(k_{\max},\cdot)$; $c=3$ suffices on all low rows for every r). Hence $f(x)\le A\big(K,\ A(k_{\max}+c,\langle x\rangle)\big)$, and (I) with $a=K$, $b=k_{\max}+c$ gives

$$f(x)\ \le\ A\big(K+k_{\max}+c+1,\ \langle x\rangle\big),$$

a fixed level, as required.

- **Primitive recursion.** Let $f(x,0)=g(x)$, $f(x,t+1)=h(x,t,f(x,t))$, with g,h PR, so $g(x)\le A(a,\langle x\rangle)$ and $h(x,t,z)\le A(b,\langle x\rangle+t+z)$ by induction. The step map $T_{x,t}(z)=h(x,t,z)$ satisfies $T_{x,t}(z)\le A(b,\ z+(\langle x\rangle+t))$, so by (II) its t-fold iterate from the base value $g(x)\le A(a,\langle x\rangle)$ obeys

$$f(x,t)\ \le\ A(b+1,\ \langle x\rangle+t)\ \le\ A(b+1,\ \langle(x,t)\rangle).$$

Thus $K=b+1$ suffices. □

Lemma 2 is the content: the Ackermann rows *exhaust* the primitive-recursive functions. No matter how a PR function is built — however many nested primitive recursions — it is bounded by one *fixed* row $A(K,\cdot)$.

> **Remark (equivalence with the FGH).** The Ackermann rows and the FGH levels have the same iterative growth rate, and dominate each other: each fixed row $A(k,\cdot)$ is itself primitive recursive (a finite construction), hence by the FGH-domination characterization is $\le F_m(\cdot)$ for some m; and each level F_m is PR by Lemma 1, hence by Lemma 2 is $\le A(k,\cdot)$ for some k. Thus Lemma 2 is equivalent to the familiar FGH-domination characterization (every PR function is $\le F_m(\langle x\rangle)$ eventually for some fixed m). We use the Ackermann-row form below because the diagonal escape (Theorem 2) is cleanest there.

> **Lemma 3 (Monotonicity).** A is strictly increasing in each argument: $A(m,n+1)>A(m,n)$ and $A(m+1,n)>A(m,n)$ for all m,n.

Proof. We prove both statements simultaneously by induction on m, the second-variable part by an inner induction on n at each m.

Base $m=0$**.** $A(0,n+1)=n+2=(n+1)+1>A(0,n)$; and $A(1,n)=n+2>(n+1)=A(0,n)$.

Inductive step. Assume both parts hold for m (so in particular $A(m,\cdot)$ and $A(m+1,\cdot)$ are strictly increasing in their second argument). We prove both parts for $m+1$.

- *Second variable.* $A(m+1,n+1)=A(m,A(m+1,n))$ and $A(m+1,n)=A(m,A(m+1,n-1))$ (with $A(m+1,0)=A(m,1)>A(m,0)$ the $n=0$ case). By the inner induction $A(m+1,n)>A(m+1,n-1)$, and since $A(m,\cdot)$ is strictly increasing (induction hypothesis), $A(m,A(m+1,n))>A(m,A(m+1,n-1))$, i.e. $A(m+1,n+1)>A(m+1,n)$.
- *First variable.* We show $A(m+2,n)>A(m+1,n)$. For $n=0$: $A(m+2,0)=A(m+1,1)>A(m+1,0)$ since $A(m+1,\cdot)$ is strictly increasing (previous bullet). For $n\ge 1$: $A(m+2,n)=A(m+1,A(m+2,n-1))$ and, by the inner induction on n, $A(m+2,n-1)>A(m+1,n-1)$; applying the strictly-increasing map $A(m+1,\cdot)$ and using $A(m+1,A(m+1,n-1))=A(m+1,n)$ gives

$$A(m+2,n)=A\big(m+1,A(m+2,n-1)\big)\ >\ A\big(m+1,A(m+1,n-1)\big)=A(m+1,n).$$

□

The proof is a simultaneous double induction: the first-variable step at $(m+1,n)$ is fed by the first-variable step at $(m+1,n-1)$ (inner induction) composed with the second-variable monotonicity of $A(m+1,\cdot)$.

---

6. A is not primitive recursive

We are now ready for the main theorem. The argument is a **diagonal escape**: the diagonal $n \mapsto A(n, n)$ climbs the Ackermann rows without bound, while Lemma 2 says no primitive-recursive function can.

> **Theorem 2.** The Ackermann function $A : \mathbb{N}^2 \to \mathbb{N}$ is total and Turing-computable, but is **not** primitive recursive.

Proof. Totality and Turing-computability are §2.2 and §3. For non-PR-ness, consider the **diagonal** $a(n) = A(n, n)$.

Assume, for contradiction, that A is PR. Then $a(n) = A(n, n)$ is PR (the diagonal of a PR function is PR — it is the composition $n \mapsto A(n, n)$). By Lemma 2 (with $k = 1$, so the size is just $\langle n \rangle = n$), there is a fixed K such that

$$a(n) = A(n, n) \;\leq\; A(K, n) \qquad \text{for all } n. \tag{7}$$

But Lemma 3 says A is strictly increasing in its *first* argument. Hence for every $n > K$, iterating the step $A(j + 1, m) > A(j, m)$ for $j = K, \ldots, n - 1$ gives $A(n, m) > A(K, m)$ for all m, in particular (taking $m = n$)

$$A(n, n) \;>\; A(K, n) \qquad \text{for all } n > K. \tag{8}$$

(8) contradicts (7) for every $n > K$. □

The contradiction is the exact mirror image of Lemma 2: Lemma 2 says any PR function is pinned to one *fixed* row $A(K, \cdot)$, whereas the diagonal $a(n) = A(n, n)$ has its *row index* equal to its input, so it outruns every fixed row as n grows.

> **Remark (why the pointwise FGH bound is the wrong tool).** It is tempting to prove Theorem 2 by bounding $A(m, n)$ pointwise between two FGH levels, $F_{m-1}(n) \leq A(m, n) \leq F_{m+1}(n)$, and then letting $m = n$. The *lower* bound is **false**: with $F_2(n) = 2^n n$ (not $2n^2$), one has $A(3, n) = 2^{n+3} - 3$, and $A(3, 8) = 2045 < 2048 = F_2(8)$, so $A(3, n)$ is *not* eventually $\geq F_2(n)$. The Ackermann row $A(3, \cdot)$ and the FGH level F_2 have the same *iterative* growth rate but different *constants*, and pointwise dominance fails. The diagonal-escape proof above avoids this pitfall entirely: it needs only the *row-index* behavior (Lemma 2) and strict monotonicity (Lemma 3), not pointwise $A(m, n)$ vs $F_m(n)$ comparisons. The FGH and the Ackermann rows are equivalent as *scales* (Remark after Lemma 2); the proof is cleanest in row form.

Corollary 2 (resource form). No Turing machine computing A has a primitive-recursive bound on its running time.

Proof. Immediate from Theorem 2 and Corollary 1. □

This is the precise sense in which the Ackermann function "outruns" the primitive-recursive world: it is not that some particular machine is slow, but that **the function itself** cannot be certified by any PR clock. The diagonal $a(n) = A(n, n)$ is the object that climbs the Ackermann rows (equivalently, the FGH levels) without bound, and no single fixed row can bound it.

---

7. Proof-theoretic significance: the threshold of Peano Arithmetic

The primitive-recursive/total-computable divide has a sharp **proof-theoretic** shadow, which is where the relationship to formal systems — and, indirectly, to the halting problem — becomes visible.

7.1 Provable totality

- Every **primitive-recursive** function is provably total in very weak arithmetic (already in fragments with only Δ_0 or Σ_1 induction): its totality is witnessed by a straightforward induction mirroring its finite construction.
- A **is provably total in Peano Arithmetic (PA).** The induction on m of §2.2, with the nested recursion handled by induction on n, is formalizable in PA.
- The **provably-total functions of PA** are exactly those dominated by the fast-growing hierarchy up to the ordinal ε_0 (the proof-theoretic ordinal of PA) — the classical ordinal-analysis result (Girard; see Hájek & Pudlák). Concretely, f is provably total in PA iff $f(n) \leq F_\alpha(n)$ for some $\alpha < \varepsilon_0$.

The diagonal $a(n) = A(n, n)$ is *not* dominated by any single fixed level F_k (that is exactly Theorem 2, restated in FGH form); rather, each value $a(n) = A(n, n)$ is dominated by some level $F_{k(n)}$ with $k(n)$ growing with n (the diagonal's scale-index is its input, per §5–§6). Since every finite level index is $< \varepsilon_0$, the totality of A — which requires induction over the whole finite initial segment of the hierarchy — is provable in PA, but not in the fragment that proves totality of the primitive-recursive functions (which only need a *fixed* level). A is thus the threshold object: provably total in PA, yet not in the PR class.

7.2 The Kirby–Paris hydra interpretation

The most vivid modern interpretation of A's growth is the **hydra game** (Kirby & Paris, 1982). A "hydra" is a finite rooted tree; a "move" consists of the hero cutting off a head, after which the hydra regrows a bounded number of copies of the stump. The game always terminates — the hydra is finite and each move decreases an associated ordinal $< \varepsilon_0$ — but the number of moves to kill a hydra of size n grows at the rate of the fast-growing hierarchy, hence like $A(n, n)$. Crucially, the statement "the hydra game always terminates" is **provable in PA but not in the weaker Δ_0-induction fragment**. This gives a *non-arithmetic*, combinatorial reason why A grows as fast as it does and why its totality lands exactly at PA's threshold: A is, up to a level shift, the function measuring the termination time of a process whose well-foundedness is provable in PA but not below it.

7.3 Relation to the halting problem

The general problem "does this Turing machine halt on **all** inputs?" (i.e. "does it compute a total function?") is **undecidable** — it is Π^0_2-complete, strictly harder than the halting problem. The Ackermann function is the *concrete, positive* counterpart to this undecidability: it is a specific function whose totality we can *prove* (in PA) and whose machine M_A we can *write down*, yet which lies beyond the primitive-recursive, PR-bounded-computation realm. A is thus the boundary between the functions a weak theory can certify as total (the PR functions) and those that are total but require the full induction of PA — the same induction strength that separates PA from its Δ_0 fragment in the hydra theorem.

---

8. Conclusion

The relationship between the Ackermann function and the Turing machine is not that the machine computes the function — that is shared by every total computable function. It is that A **is the canonical, minimal witness to the strictness of the inclusion**

$$\text{primitive recursive} \subsetneq \text{total Turing-computable},$$

and it realizes that strictness in three equivalent, precisely-stated ways:

1. **Functionally:** A is total and Turing-computable (§3) but not primitive recursive (§6): the diagonal $n \mapsto A(n, n)$ has its scale-index equal to its input and so outruns ev-

ery fixed element of the scale (Ackermann rows / FGH levels) that exhausts the PR functions (Lemma 2 + Lemma 3).
2. **Resource-wise:** by Theorem 1 and Corollary 2, A is Turing-computable but **no Turing machine computing it has a primitive-recursive time bound**. A is the first total function a machine can evaluate that no PR clock can certify.
3. **Proof-theoretically:** A is provably total in PA and measures the termination of the hydra game; proving its totality requires induction over the whole finite initial segment of the hierarchy up to ε_0 — the full strength of PA's provable totality — whereas the PR functions need only a fixed level.

Each increment of A's first argument adds one iteration level — successor, addition, multiplication, exponentiation, tetration — and it is precisely this unbounded ascent through the exhaustion scale (the fast-growing hierarchy, or equivalently the Ackermann rows), still *within* it, that places A on the boundary between what a Turing machine can compute with provably bounded effort and what it can compute at all.

References

- **Ackermann, J.** (1928). *Zum Hilbertschen Aufbau der reellen Zahlen.* Mathematische Annalen **107**, 459–488.
- **Péter, R.** (1956–58). *Ein kombinatorischer Satz über die Funktionenklasse der primitiv-rekursiven Funktionen.* Acta Physico-Mathematicica **6**, 1–10; see also *Die Theorie der rekursiven Funktionen* (Akadémiai Kiadó, 1956).
- **Goodstein, R. L.** (1947). *On the computable functions.* Journal of Symbolic Logic **12**, 127–131.
- **Kleene, S. C.** (1936). *General form of recursive function.* American Journal of Mathematics **57**, 254–263.
- **Church, A.** (1936). *An unsolvability problem in arithmetic.* Journal of the London Mathematical Society **11**, 218–228.
- **Turing, A. M.** (1936). *On computable numbers, with an application to the Entscheidungsproblem.* Proceedings of the London Mathematical Society **42**, 230–265.
- **Rogers, H.** (1967). *Theory of Recursive Functions and Effective Computability.* MIT Press.
- **Cutland, N. J.** (1980). *Computability: An Introduction to Recursive Function Theory.* Cambridge University Press.
- **Hájek, P., & Pudlák, P.** (1993). *Metamathematics of First-Order Arithmetic.* Cambridge University Press. (Provably-total functions of PA; ordinal ε_0.)
- **Kirby, L., & Paris, J.** (1982). *Accessible independence results for Peano arithmetic.* Bulletin of the London Mathematical Society **14**(4), 285–293. (The hydra game.)
- **Odifreddi, P. E.** (1992). *Classical Recursion Theory.* North-Holland.